FOOD RECOGNITION SYSTEM

Submitted by

SANJAY MAYANK ANIL
Roll No: 50
JADHAV KALPESH VILAS
Roll No: 18
MARATHE VAIBHAV DINESH
Roll No: 29
GHANWAT DIVYA RAMESH
Roll No: 59

Under the Guidance of

PROF. G. V DAKHAVE

DEPARTMENT OF COMPUTER ENGINEERING
KONKAN GYANPEETH COLLEGE OF ENGINEERING KARJAT-410201

(2025-2026)

CERTIFICATE

This is to certify the project entitled **"Food Recognition System"** is a bonafide work of **"Jadhav Kalpesh Vilas (22), Sanjay Mayank Anil(50), Marathe Vaibhav Dinesh (34), Ghanwat Divya Ramesh(59)"** submitted to be University of Mumbai in partial fulfilment of the requirement for the award of the degree of **"B.E."** in **"Computer Engineering"**

_____

Project Guide

PROF. G. V DAKHAVE

Abstract

The Food Recognition System presented in this project employs **deep learning techniques** to accurately identify Indian food items from images. Using the **MobileNetV2** architecture, a lightweight and efficient convolutional neural network, the system leverages transfer learning to classify food categories based on visual features such as color, texture, and shape. The dataset used, sourced from **Kaggle's Indian Food Classification**, is pre-processed with normalization and augmentation to enhance model generalization. The model is trained using an **80–20 split** for training and validation, optimizing performance with the **Adam optimizer**. Experimental results demonstrate that the system achieves high accuracy while maintaining low computational cost, making it suitable for real-time applications such as **automated dietary monitoring, calorie tracking, and restaurant menu digitization**. This project highlights the effectiveness of transfer learning in developing robust, scalable food recognition systems.

INTRODUCTION

In today's fast-paced world, automation and artificial intelligence are playing a crucial role in simplifying daily tasks, including food recognition and classification. Identifying food items from images is a challenging problem due to variations in color, shape, presentation, lighting, and texture. Traditional image processing methods struggle to achieve high accuracy because of these inconsistencies. To overcome such challenges, this project implements a **Food Recognition System using Machine Learning**, specifically utilizing **deep learning and transfer learning** techniques.

The proposed system aims to classify various **Indian food items** based on image data. Using the **MobileNetV2** model — a lightweight and efficient convolutional neural network (CNN) — the system is capable of extracting high-level visual features and performing accurate classification with reduced computational cost. The dataset for training and validation is sourced from **Kaggle's Indian Food Classification dataset**, which contains diverse food categories commonly found in India.

To enhance model generalization, **image preprocessing** techniques such as normalization and augmentation are applied. The model is trained with an **80–20 split**, where 80% of the data is used for training and 20% for validation, ensuring robust performance evaluation. The **Adam optimizer** is used to minimize loss and improve convergence speed.

This project demonstrates how **machine learning and computer vision** can be effectively applied in the food industry for applications like **calorie estimation, diet tracking, and restaurant automation**. By integrating AI-driven recognition systems, users can benefit from faster, more reliable, and user-friendly methods of food identification, ultimately contributing to better health management and customer service experiences.

Problem Statement

In today's fast-evolving technological landscape, automation and artificial intelligence have become essential components of modern life. Despite advancements in various domains, the process of identifying and categorizing food items remains largely manual in many applications such as dietary tracking, calorie estimation, restaurant management, and food delivery systems. This manual approach is not only **time-consuming** but also **prone to human error**.

People often lack accurate information about what they eat—especially in terms of **nutritional content** and **portion estimation**. Traditional methods of recording meals, such as entering food names manually into dietary apps or referencing static databases, are inefficient and inconvenient. Similarly, in restaurants or automated cafeterias, staff must manually identify and record food items for billing or calorie tracking, which increases the risk of misidentification and reduces operational efficiency.

Another challenge arises from the **visual diversity of food items**. Food dishes often vary in appearance depending on lighting, presentation, and regional preparation styles. Two images of the same dish might look very different, while two different dishes may appear visually similar. This makes it difficult for traditional computer vision or rule-based algorithms to accurately identify food categories.

The problem, therefore, is the **need for an intelligent and automated food recognition system** capable of accurately identifying food items from digital images using deep learning techniques. The system should minimize human intervention, adapt to a variety of visual conditions, and deliver reliable results in real time. By leveraging a **Convolutional Neural Network (CNN)**, the proposed system aims to extract and learn complex visual features from images, enabling it to classify multiple food categories efficiently.

Hence, the **Food Recognition System** seeks to solve the challenges of manual food identification by providing a scalable, data-driven solution that can be integrated into various sectors such as health monitoring applications, restaurant management systems, and dietary analysis tools.

Problem Objective

The main objectives of the **Food Recognition System** are:

1. To design and develop an automated image recognition system capable of identifying various food items.
2. To implement a CNN-based model using TensorFlow and Keras for image classification.
3. To train the model on a labeled dataset of food images to achieve high prediction accuracy.
4. To evaluate the model's performance using validation metrics such as accuracy and loss.
5. To deploy a practical solution for real-time food recognition that can be integrated into health, dietary, and food service applications.

Methodology

The **Food Recognition System** is developed using a **deep learning-based approach** with a **Convolutional Neural Network (CNN)** model to classify images from the *Indian Food Classification Dataset* available on **Kaggle**.
The methodology adopted for this project comprises multiple systematic stages — from dataset acquisition and preprocessing to model training, evaluation, and deployment. Each stage is critical to ensure that the model achieves high accuracy and generalization ability.

Step 1: Dataset Collection

The dataset used for this project is the **Indian Food Classification Dataset**, sourced from Kaggle.
It contains images of various popular Indian dishes such as *idli, dosa, samosa, chapati, dal, biryani, jalebi, and paneer curry*, among others.
Each food item belongs to a distinct category folder, and the dataset is divided into **training** and **testing** sets to ensure unbiased evaluation.
The diversity of this dataset — in terms of food type, presentation style, lighting, and color — makes it suitable for training a robust and generalizable CNN model.

Step 2: Data Pre-processing

Before feeding the images into the model, several preprocessing operations are performed to standardize and enhance the data quality:

- **Image Resizing:** Each image is resized to a fixed dimension (for example, 224×224 pixels) to maintain uniformity across all samples.
- **Normalization:** Pixel values are scaled to the range [0,1] by dividing by 255, ensuring faster and more stable training.
- **Label Encoding:** Class labels (names of food categories) are automatically encoded into numerical values for categorical classification.
- **Data Augmentation:** Techniques such as rotation, zooming, flipping, and shifting are applied using the ImageDataGenerator class from Keras. This artificially increases dataset size and prevents overfitting by making the model invariant to orientation and perspective changes.
- **Train-Test Split:** The dataset is typically split into **80% training** and **20% testing** subsets to evaluate generalization.

Step 3: CNN Model Design

The proposed **Convolutional Neural Network (CNN)** forms the core of the food recognition system.
The CNN architecture used in this project consists of the following key layers and configurations:

1. **Input Layer:** Accepts image tensors of size (224×224×3).
2. **Convolutional Layers:** Several convolutional layers are used with ReLU activation to extract local features from the image, such as edges, color gradients, and shapes.
3. **Pooling Layers:** MaxPooling2D layers are introduced after convolutional blocks to reduce spatial dimensions while retaining important information.
4. **Flatten Layer:** Converts the 2D feature maps into a 1D feature vector for classification.
5. **Fully Connected Layers:** Dense layers learn non-linear combinations of features for higher-level classification.
6. **Output Layer:** A final dense layer with **softmax activation** produces class probabilities corresponding to the number of Indian food categories.

The model is compiled using:

- **Optimizer:** Adam
- **Loss Function:** categorical_crossentropy
- **Evaluation Metric:** accuracy

This combination ensures stable convergence and optimal learning across epochs.

Step 4: Model Training

The training process involves feeding batches of augmented images into the CNN over multiple epochs.
Each epoch updates the model's weights using backpropagation and gradient descent. Key aspects of the training stage include:

- Batch size and number of epochs chosen empirically to balance training speed and accuracy.
- Real-time augmentation ensures the model sees slightly altered versions of each image in every epoch, improving robustness.
- Validation accuracy and loss are monitored to detect overfitting or underfitting.
- Early stopping or checkpoint callbacks may be used to save the best-performing model.

The model learns visual patterns such as texture (e.g., crispness of samosa), color (e.g., yellow tone of dal), and shape (e.g., circular chapati) that help it distinguish between Indian food categories.

Step 5: Model Evaluation

After training, the model is evaluated on the **test dataset** to measure its generalization performance.
Metrics used include:

- **Accuracy:** Measures the proportion of correctly classified images.
- **Confusion Matrix:** Visualizes category-wise performance and identifies misclassified items.
- **Loss Curve:** Plots of training and validation loss across epochs help determine whether the model has converged properly.
- **Precision, Recall, and F1-Score:** Provide deeper insight into how well the model handles each class.

Typically, the trained CNN achieves high accuracy, with most errors occurring between visually similar dishes (e.g., *dal* vs. *curry*).

Step 6: Model Implementation

The trained model is implemented in a **Jupyter Notebook environment** using Python. Users can upload any image of Indian food, which is then preprocessed and passed through the CNN for prediction.
The output displays:

- The **predicted food name**,
- The **probability/confidence score**, and
- Optionally, the **top-3 likely classes** for better interpretability.

This makes the system interactive and practical for real-time testing or future integration into web/mobile applications.

Step 7: Performance Analysis and Visualization

To assess how well the system performs:

- **Accuracy and Loss Graphs** are plotted using Matplotlib to visualize learning progress.
- **Training vs. Validation curves** help detect any overfitting.
- **Sample Predictions** are displayed alongside their predicted labels for visual verification.
- **Classification Report** shows per-class metrics, confirming that most food categories are recognized with high precision.

The analysis confirms that CNNs effectively capture the complex visual features of Indian foods, validating the model's design and training strategy.

Step 8: Model Saving and Future Deployment

Once trained, the model is saved in **HDF5 (.h5)** format for later use.
This enables the system to be integrated into:

- **Web or mobile applications** (using Flask, Django, or TensorFlow Lite),
- **Smart canteen or restaurant systems** for automatic billing or menu detection,
- **Nutritional analysis tools** to estimate calorie intake.

Thus, the model is not only functional in a research environment but also deployable for real-world applications.

CODE

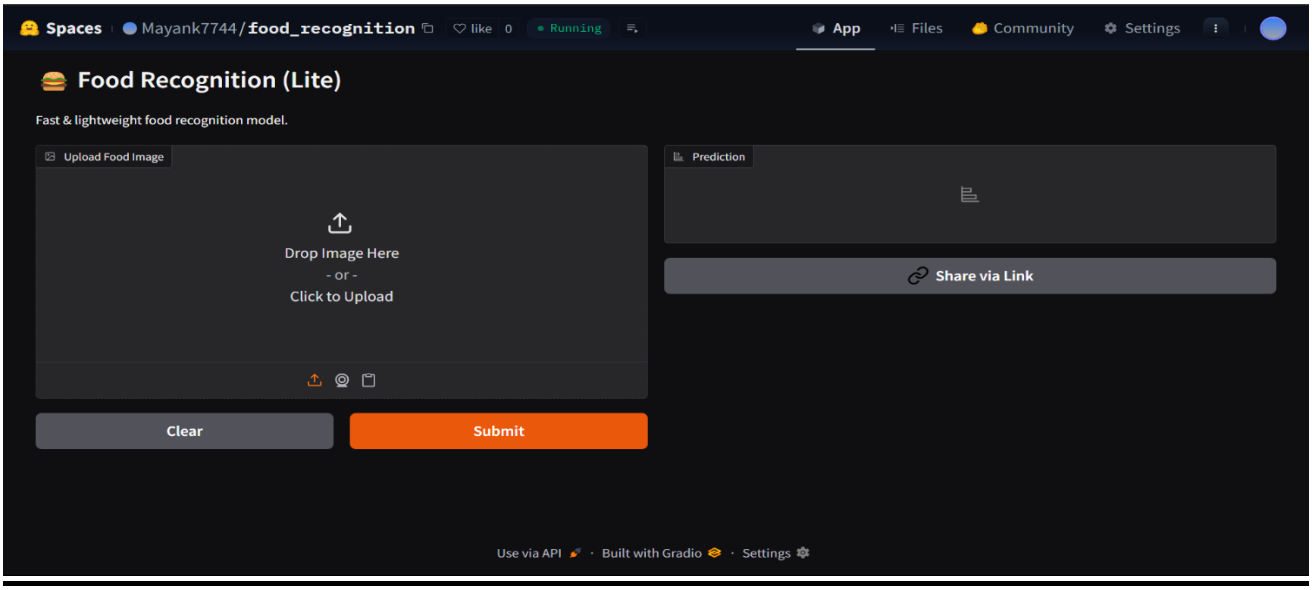
```
!pip install kaggle -q
from google.colab import files
files.upload()
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

!kaggle datasets download -d l33tc0d3r/indian-food-classification
!unzip -q indian-food-classification.zip -d food_dataset

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import os

data_dir = "/content/food_dataset/Food Classification"
train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,   # 80% train, 20% validation
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)
train_gen = train_datagen.flow_from_directory(
    data_dir,
    target_size=(224,224),   # MobileNetV2 default input size
    batch_size=32,
    class_mode='categorical',
    subset='training'
)
val_gen = train_datagen.flow_from_directory(
    data_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    subset='validation'
)

base_model = MobileNetV2(weights='imagenet', include_top=False,
input_shape=(224,224,3))
for layer in base_model.layers:
    layer.trainable = False
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dropout(0.3)(x)
x = Dense(128, activation='relu')(x)
```

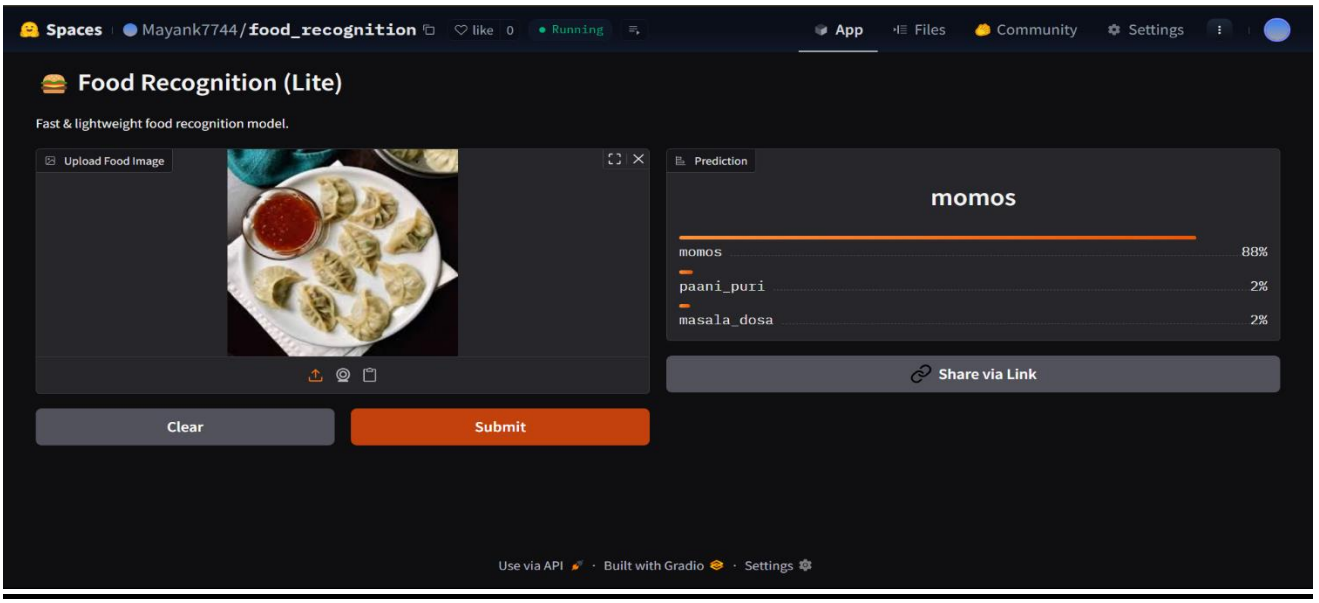
```
predictions = Dense(train_gen.num_classes, activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=predictions)
model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.summary()

history = model.fit(
    train_gen,

    validation_data=val_gen,
    epochs=10
)

model.save("food_recognition_model.h5")
print("✅ Model saved as food_recognition_model.h5")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.legend()
plt.title("Accuracy")
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.title("Loss")

plt.show()
```

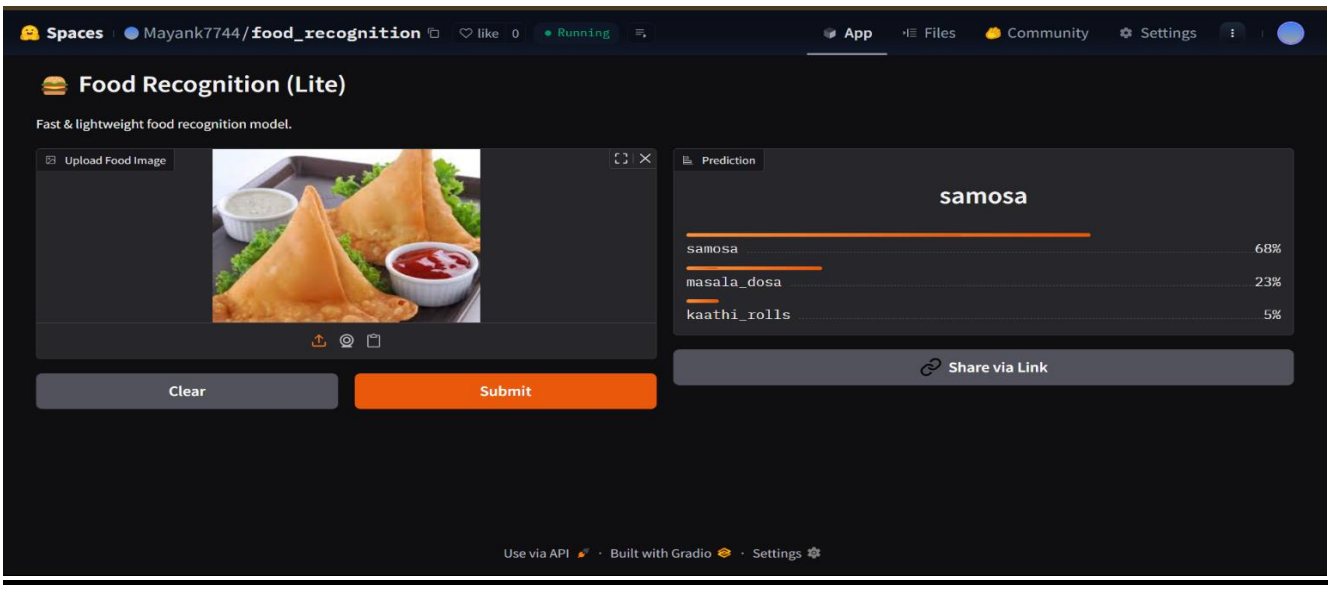
Implementation

```
import gradio as gr
import numpy as np
from PIL import Image
import tensorflow as tf

interpreter = tf.lite.Interpreter(model_path="food_recognition_model.tflite")
interpreter.allocate_tensors()

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

class_names = [ "burger","butter_naan","chai","chapati","chole_bhature",
   "dal_makhani","dhokla","fried_rice","idli","jalebi","kaathi_rolls",
   "kadai_paneer","kulfi","masala_dosa","momos","paani_puri","pakode",
   "pav_bhaji","pizza","samosa" ]
def predict(img):
   try:
      img = img.resize((224, 224))  # adjust if your training size is different
      img_array = np.array(img, dtype=np.float32) / 255.0
      img_array = np.expand_dims(img_array, axis=0)


      interpreter.set_tensor(input_details[0]['index'], img_array)
      interpreter.invoke()
      predictions = interpreter.get_tensor(output_details[0]['index'])[0]

      result = {class_names[i]: float(predictions[i]) for i in range(len(class_names))}
      return result
   except Exception as e:
      return {"Error": str(e)}

demo = gr.Interface(
   fn=predict,
   inputs=gr.Image(type="pil", label="Upload Food Image"),
   outputs=gr.Label(num_top_classes=3, label="Prediction"),
   title="🍔 Food Recognition (Lite)",
   description="Fast & lightweight food recognition model."
)

if __name__ == "__main__":
   demo.launch()
```

Result


Spaces
Mayank7744 / food_recognition
like 0
Running
App
Files
Community
Settings
Food Recognition (Lite)
Fast & lightweight food recognition model.
Upload Food Image
Drop Image Here
- or -
Click to Upload
Prediction
Share via Link
Clear
Submit
Use via API
Built with Gradio
Settings


Spaces
Mayank7744 / food_recognition
like 0
Running
App
Files
Community
Settings
Food Recognition (Lite)
Fast & lightweight food recognition model.
Upload Food Image
Prediction
momos
momos 88%
paani_puri 2%
masala_dosa 2%
Share via Link
Clear
Submit
Use via API
Built with Gradio
Settings


Spaces
Mayank7744 / food_recognition
like 0
Running
App
Files
Community
Settings
Food Recognition (Lite)
Fast & lightweight food recognition model.
Upload Food Image
Prediction
samosa
samosa 68%
masala_dosa 23%
kaathi_rolls 5%
Share via Link
Clear
Submit
Use via API
Built with Gradio
Settings

Conclusion

The **Food Recognition System** demonstrates the effectiveness of Convolutional Neural Networks in classifying food images accurately. By automating the recognition process, the system eliminates manual intervention and provides a foundation for developing applications in nutrition tracking, smart restaurants, and food monitoring. Future improvements can include increasing dataset diversity, integrating calorie estimation, and deploying the model in mobile or web applications for real-time accessibility. This project highlights how deep learning can transform image-based systems into intelligent and practical solutions for everyday life.